

CS633 Assignment Group Number 38

Arfa Khayal(251110013) Divakar Bajpai(251110019) Nishant(251110053)
Shubham Rawat(251110070) Pujari Krishnachandu (220832)

1 Code Description

The program performs iterative distance-based pairwise communication using two communication distances ($D1$ and $D2$), applies mathematical transformations to exchanged data, and computes global maximum values at the end of execution.

1.1 Input Parameters

The program requires five command-line arguments:

- M : Number of elements to process of type double.
- $D1$: First communication distance.
- $D2$: Second communication distance.
- T : Number of iterations.
- $seed$: Random seed for initialization.

1.2 Memory Allocation

Each process dynamically allocates:

- `bufferUpdatedForD1[M]` – Working buffer for $D1$ communication.
- `bufferUpdatedForD2[M]` – Working buffer for $D2$ communication.
- `buffer[M]` – Temporary computation buffer.
- `dataReceivedByD1[M]`, `dataReceivedFromD1[M]`.
- `dataReceivedByD2[M]`, `dataReceivedFromD2[M]`.
- `finalMaxValues[2]` – Stores local maxima.

1.3 Initialization

Each process initializes its local buffers using:

```
bufferUpdatedForD1[i] =  
    (double)rand() * (rank + 1) / 10000.0;
```

The data is rank-dependent to ensure variation across processes. The $D2$ buffer is initialized as a copy of the $D1$ buffer.

1.4 Iterative Communication and Computation

The core computation is executed inside:

```
for (int iter = 0; iter < T; iter++)
```

Each iteration consists of two batches:

- **Batch 1: Distance $D1$**
- **Batch 2: Distance $D2$**

Each batch contains two phases to ensure bidirectional communication.

1.4.1 Batch 1: Distance $D1$

Process roles are determined using:

```
(rank / D1) % 2
```

Phase 1

- Sender processes send `bufferUpdatedForD1` to `rank + D1`.
- Receiver processes:
 - Receive data.
 - Square each element.
 - Send squared data back.
- Sender updates its buffer using:

```
bufferUpdatedForD1[i] =  
    (unsigned long long)value % 100000;
```

Phase 2 Sender and receiver roles are reversed and the same computation pattern is repeated.

1.4.2 Batch 2: Distance $D2$

The same two-phase structure is applied using distance $D2$.

Receiver Computation

```
buffer[i] = log(dataReceivedByD2[i]);
```

Sender Update

```
bufferUpdatedForD2[i] =  
    dataReceivedFromD2[i] * 100000;
```

1.5 Timing Measurement

Execution time is measured using `MPI.Wtime()`. The timer starts before the main iterative loop and stops after completing all T iterations and the final reduction at rank 0.

The reported time includes all communication and computation phases, excluding initialization and memory allocation.

1.6 Local Maximum Computation and Global Reduction

After completing all iterations, each process computes the maximum values of `bufferUpdatedForD1` and `bufferUpdatedForD2` using a helper function that performs a linear scan over the array.

A manual reduction is implemented. Non-zero ranks send their local maxima to rank 0, which sequentially receives the values and determines the global maxima using `fmax`.

1.7 Final Output

Rank 0 prints:

```
max_val_at_D1  max_val_at_D2  executionTime
```

Where:

- First value: Global maximum for $D1$
- Second value: Global maximum for $D2$
- Third value: Total execution time

Diagrammatic Representation

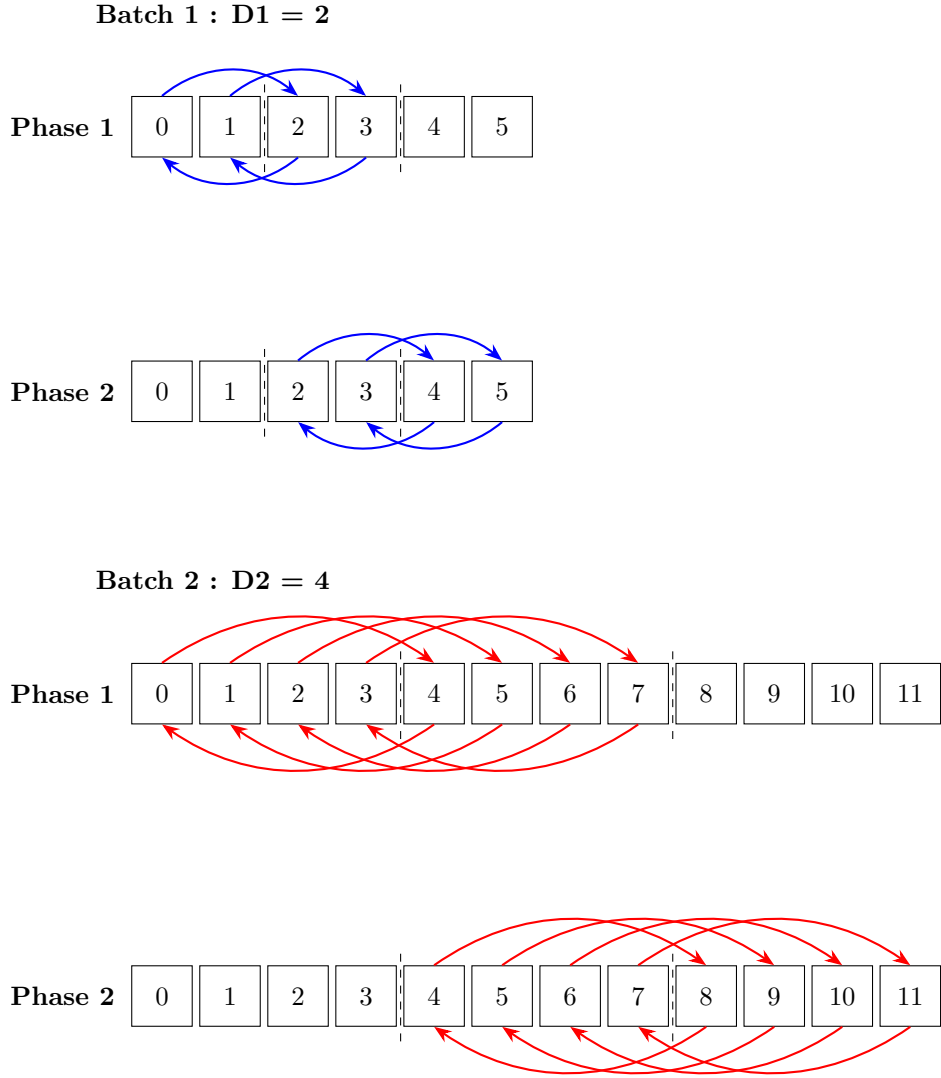


Figure 1: Two communication batches. Batch 1 uses $D1 = 2$ and Batch 2 uses $D2 = 4$. Each batch consists of two phases with bidirectional communication between paired ranks. Dashed vertical lines indicate cluster boundaries.

2 Code Compilation and Execution Instructions

2.1 Running on the Param Rudra SSH Server

First, log in to the server using the credentials provided via email.
Compile the code using:

```
mpicc -o src src.c -lm
```

Then submit the job using:

```
sbatch submit<i>.sh
```

where $i \in \{1, 2, 3, 4, 5, 6\}$. The total number of configurations is 6, given by:

$(P, M) \in \{(8, 262144), (8, 1048576), (16, 262144), (16, 1048576), (32, 262144), (32, 1048576)\}$.

Each configuration was executed 5 times.

2.2 Running on a Local Machine

Compile the code using:

```
mpicc -o src src.c -lm
```

Run the program using:

```
mpirun --oversubscribe -np P ./src M D1 D2 T seed
```

Example:

```
mpirun --oversubscribe -np 32 ./src 1048576 2 4 10 1000
```

The `--oversubscribe` flag is optional and depends on the machine.
The `submit.sh` script and `src.c` is provided in the submitted `Group38.zip` folder.

3 Results

3.1 Configuration

The execution was performed using the following configuration:

- **Computing Environment:**

- 16 processes per node
- 2 compute nodes

- **Process counts tested:**

$$P \in \{8, 16, 32\}$$

- **Data sizes:**

$$M \in \{262144, 1048576\} \text{ doubles}$$

- **Parameters:**

- Communication distances: $D_1 = 2, D_2 = 4$
- Number of iterations: $T = 10$
- Random seed: 1000

- **Repetitions:**

- Each configuration executed 5 times

3.2 Execution Time Results

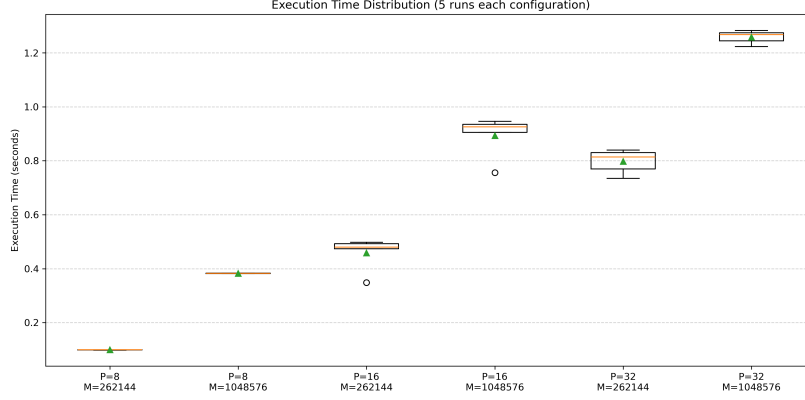


Figure 2: Boxplots of execution time for process counts $P = 8, 16$, and 32 with data sizes $M = 262144$ and $M = 1048576$, based on five runs per configuration.

Figure 2 illustrates execution time versus process count. The x-axis denotes the number of processes (P) and the y-axis denotes execution time in seconds. For $P = 8, 16$, and 32 , separate boxplots represent $M = 262144$ and $M = 1048576$. Execution time increases with both process count and data size, reflecting higher communication volume and synchronization overhead.

Process (P)	M	D1_max	D2_max	Avg Time (s)
8	262144	99936	1416360.08158	0.09923
8	1048576	99936	1416360.08158	0.38282
16	262144	99936	1416360.08158	0.45861
16	1048576	99936	1416360.08158	0.89347
32	262144	99936	1416360.08158	0.79761
32	1048576	99936	1416360.08158	1.25859

Table 1: Average execution time over five runs

Table 1 presents the average execution time for each configuration over five runs. The data shows a consistent increase in runtime with increasing process count and data size, indicating the growing impact of communication and synchronization overhead.

3.3 Analysis

Efficiency Trend and Communication behaviour

Efficiency decreases as the number of processes increases. Communication and synchronization overhead outweigh the computational benefits, resulting in diminishing performance gains.

- **Intensive Message Exchange and Lightweight Computation:** Each iteration requires multiple message exchanges across the network. The computation performed per data element is minimal, meaning the CPU completes its tasks faster than new data can be received.
- **Dominant Communication Overheads:** Communication costs dominate total runtime, creating an unfavorable communication-to-computation ratio and limiting scalability.
- **Blocking MPI Operations:** The implementation relies on `MPI_Send` and `MPI_Recv`, which are synchronous blocking calls. Because these operations are blocking, a process cannot proceed until the message transfer completes.
- **Idle Runtime and Propagation:** A significant portion of execution time is spent waiting rather than computing. Delays in one process propagate to others due to synchronization, reducing overall resource utilization.

Impact of Data Size on Execution Time

Execution time increases consistently with larger data size (M). This behavior is directly related to increased communication volume, computation workload.

- **Linear Growth in Communication Volume:** Each iteration involves sending and receiving buffers of size M doubles. Increasing M directly increases the amount of data transmitted over the network, resulting in longer message transfer times.
- **Increased Computational Workload:** All receiver-side and sender-side computations are performed element-wise over M elements. A larger data size increases the total number of arithmetic operations per iteration.
- **Synchronization Delays:** Since blocking MPI communication is used, larger message sizes prolong the time each process waits for communication to complete. This increases synchronization delays across iterations.
- **Network Contention Effects:** When multiple processes exchange larger messages simultaneously, network congestion may increase (depends on links and topology), further extending communication time.
- **Proportional Runtime Scaling:** The observed results show that increasing M results in a proportional increase in total runtime, indicating that both computation and communication scale with data size.

4 Conclusions

The implemented distance-based pairwise communication algorithm, executed in two batches ($D1$ and $D2$) with two-phase bidirectional exchanges, is predominantly communication-bound. As the number of processes increases, parallel efficiency decreases due to frequent message passing and blocking MPI operations that introduce synchronization delays and idle time. Execution time also scales proportionally with data size (M), as both communication volume and per-element computations increase.

All five group members contributed equally.